Object oriented programming with Java Lab

1. **Develop a JAVA program to demonstrate the precedence and associativity among arithmetic operators. The program should also demonstrate how the default precedence can be overridden.**

```
import java.util.Scanner;
public class ArithmeticPrecedence {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("=== ARITHMETIC OPERATOR PRECEDENCE ===\n");

      System.out.print("Enter a: ");
      int a = sc.nextInt();
      System.out.print("Enter b: ");
      int b = sc.nextInt();
      System.out.print("Enter c: ");
      int c = sc.nextInt();

      System.out.println("\nValues: a=" + a + ", b=" + b + ", c=" + c);
      System.out.println("----------------------------------------");

      // Default precedence
      System.out.println("1. a + b * c = " + (a + b * c));
      System.out.println("   (* has higher precedence than +)");

      // Overriding precedence
      System.out.println("2. (a + b) * c = " + ((a + b) * c));
      System.out.println("   (Parentheses override default precedence)");

      // Associativity
      System.out.println("3. a - b - c = " + (a - b - c));
      System.out.println("   (Left to right associativity)");

      // Mixed operators
      System.out.println("4. a + b * c - a / b = " + (a + b * c - a / b));

      sc.close();
   }
}
```

2. **Write a JAVA program to validate a date. The program should accept day, month and year and it should report whether they form a valid date or not.**

```
import java.util.Scanner;

public class DateValidator {
    public static boolean isLeapYear(int year) {
        return (year % 400 == 0) || (year % 4 == 0 && year % 100 != 0);
    }

    public static boolean isValidDate(int day, int month, int year) {
        if (year < 1 || month < 1 || month > 12 || day < 1)
            return false;

        int[] days = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

        if (month == 2 && isLeapYear(year))
            return day <= 29;

        return day <= days[month - 1];
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("=== DATE VALIDATION ===\n");

        System.out.print("Enter day: ");
        int day = sc.nextInt();
        System.out.print("Enter month: ");
        int month = sc.nextInt();
        System.out.print("Enter year: ");
        int year = sc.nextInt();

        System.out.println("\nDate: " + day + "/" + month + "/" + year);

        if (isValidDate(day, month, year)) {
            System.out.println("✓ VALID DATE");
            if (isLeapYear(year))
                System.out.println("→ " + year + " is a leap year");
        } else {
            System.out.println("✗ INVALID DATE");
        }
```

```
            sc.close();
        }
    }
```

3.Write a JAVA program to display the following pattern (center aligned):

1

2 2

3 3 3

4 4 4 4

5 5 5 5 5

```
public class NumberPattern {
  public static void main(String[] args) {
      System.out.println("=== NUMBER PATTERN ===\n");

      int rows = 5;

      for (int i = 1; i <= rows; i++) {
          // Print spaces for center alignment
          for (int j = rows; j > i; j--) {
              System.out.print(" ");
          }

          // Print numbers
          for (int j = 1; j <= i; j++) {
              System.out.print(i + " ");
          }
          System.out.println();
      }
  }
}
```

4.Write a JAVA program to print the first n members of Fibonacci series.

```
import java.util.Scanner;
public class FibonacciSeries {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.println("=== FIBONACCI SERIES ===\n");

      System.out.print("Enter number of terms: ");
      int n = sc.nextInt();

      System.out.print("Fibonacci Series: ");

      int a = 0, b = 1;

      if (n >= 1) System.out.print(a + " ");
      if (n >= 2) System.out.print(b + " ");

      for (int i = 3; i <= n; i++) {
         int next = a + b;
         System.out.print(next + " ");
         a = b;
         b = next;
      }
      System.out.println();

      sc.close();
   }
}
```

5.Write a program to generate the multiplication tables of a range of numbers between m and n inclusive and m < n.

```
import java.util.Scanner;

public class MultiplicationTable {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.println("=== MULTIPLICATION TABLES ===\n");

      System.out.print("Enter m: ");
      int m = sc.nextInt();
```

```
        System.out.print("Enter n: ");
        int n = sc.nextInt();

        if (m >= n) {
            System.out.println("Error: m must be less than n");
            sc.close();
            return;
        }

        for (int num = m; num <= n; num++) {
            System.out.println("\nTable of " + num + ":");
            for (int i = 1; i <= 10; i++) {
                System.out.println(num + " × " + i + " = " + (num * i));
            }
        }

        sc.close();
    }
}
```

6.Write a JAVA program to define a class, define instance methods for setting and retrieving values of instance variables and instantiate its object.

```
import java.util.Scanner;

class Student {
    private int rollNo;
    private String name;
    private double marks;

    public void setData(int rollNo, String name, double marks) {
        this.rollNo = rollNo;
        this.name = name;
        this.marks = marks;
    }

    public void display() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
        System.out.println("Marks: " + marks);
    }
}
```

```
public class ClassWithInstanceMethods {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("=== CLASS WITH INSTANCE METHODS ===\n");

        Student s = new Student();

        System.out.print("Enter Roll No: ");
        int roll = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Marks: ");
        double marks = sc.nextDouble();

        s.setData(roll, name, marks);

        System.out.println("\nStudent Details:");
        s.display();

        sc.close();
    }
}
```

7.Write a JAVA program to demonstrate static member data and static member methods.

```
class Employee {
    private int id;
    private String name;
    private static int count = 0;

    public Employee(int id, String name) {
        this.id = id;
        this.name = name;
        count++;
    }

    public void display() {
        System.out.println("ID: " + id + ", Name: " + name);
    }
```

```
    public static void showCount() {
        System.out.println("Total Employees: " + count);
    }
}

public class StaticMemberDemo {
    public static void main(String[] args) {
        System.out.println("=== STATIC MEMBER DEMONSTRATION ===\n");

        Employee.showCount();

        Employee e1 = new Employee(101, "Alice");
        Employee e2 = new Employee(102, "Bob");

        e1.display();
        e2.display();

        Employee.showCount();
    }
}
```

8.Write a JAVA Program to demonstrate nested classes.

```
class Outer {
    private String msg = "Hello from Outer";

    class Inner {
        public void display() {
            System.out.println(msg);
        }
    }

    static class StaticNested {
        public void display() {
            System.out.println("Hello from Static Nested");
        }
    }
}

public class NestedClassesDemo {
    public static void main(String[] args) {
        System.out.println("=== NESTED CLASSES ===\n");
```

```
        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner();
        inner.display();

        Outer.StaticNested nested = new Outer.StaticNested();
        nested.display();
    }
}
```

9.Write a JAVA program to demonstrate dynamic method dispatch.

```
class Animal {
    public void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    public void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    public void sound() {
        System.out.println("Cat meows");
    }
}

public class DynamicMethodDispatchDemo {
    public static void main(String[] args) {
        System.out.println("=== DYNAMIC METHOD DISPATCH ===\n");

        Animal ref;

        ref = new Dog();
        ref.sound();

        ref = new Cat();
        ref.sound();
    }
}
```

10.Write a JAVA program to implement inheritance and demonstrate use of method overriding.

```
class Vehicle {
   public void start() {
      System.out.println("Vehicle started");
   }
}

class Car extends Vehicle {
   public void start() {
      System.out.println("Car started with key");
   }
}

class Bike extends Vehicle {
   public void start() {
      System.out.println("Bike started with kick");
   }
}

public class InheritanceOverridingDemo {
   public static void main(String[] args) {
      System.out.println("=== INHERITANCE AND OVERRIDING ===\n");

      Vehicle v1 = new Car();
      Vehicle v2 = new Bike();

      v1.start();
      v2.start();
   }
}
```

11.Write a JAVA program to implement the concept of importing classes from user defined package and creating packages.

File: mypackage/Calculator.java

```
package mypackage;
public class Calculator {
   public int add(int a, int b) {
      return a + b;
   }
```

```
    public int sub(int a, int b) {
        return a - b;
    }
}
```

File: PackageDemo.java

```
import mypackage.Calculator;

public class PackageDemo {
    public static void main(String[] args) {
        System.out.println("=== USER DEFINED PACKAGE ===\n");

        Calculator calc = new Calculator();

        System.out.println("Addition: " + calc.add(10, 5));
        System.out.println("Subtraction: " + calc.sub(10, 5));
    }
}
```

12.Write a program to demonstrate abstract class and abstract methods.

```
abstract class Shape {
    abstract double area();

    public void display() {
        System.out.println("Area: " + area());
    }
}

class Circle extends Shape {
    double radius;

    Circle(double r) {
        radius = r;
    }

    double area() {
        return 3.14 * radius * radius;
    }
}

class Rectangle extends Shape {
    double l, b;
```

```
    Rectangle(double l, double b) {
        this.l = l;
        this.b = b;
    }

    double area() {
        return l * b;
    }
}

public class AbstractDemo {
    public static void main(String[] args) {
        System.out.println("=== ABSTRACT CLASS ===\n");

        Circle c = new Circle(5);
        Rectangle r = new Rectangle(4, 6);

        c.display();
        r.display();
    }
}
```

13.Write a JAVA Program to implement an array of objects of a class.

```
import java.util.Scanner;

class Student {
    int roll;
    String name;

    Student(int roll, String name) {
        this.roll = roll;
        this.name = name;
    }

    void display() {
        System.out.println(roll + " - " + name);
    }
}

public class ArrayOfObjects {
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);

        System.out.println("=== ARRAY OF OBJECTS ===\n");

        System.out.print("Enter number of students: ");
        int n = sc.nextInt();

        Student[] students = new Student[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Roll: ");
            int roll = sc.nextInt();
            sc.nextLine();
            System.out.print("Name: ");
            String name = sc.nextLine();
            students[i] = new Student(roll, name);
        }

        System.out.println("\nStudent List:");
        for (int i = 0; i < n; i++) {
            students[i].display();
        }

        sc.close();
    }
}
```

14.Write a JAVA Program to demonstrate String class and its methods.

```
public class StringMethodsDemo {
    public static void main(String[] args) {
        System.out.println("=== STRING METHODS ===\n");

        String s1 = "Hello";
        String s2 = "World";

        System.out.println("Length: " + s1.length());
        System.out.println("Concatenation: " + s1.concat(" " + s2));
        System.out.println("Uppercase: " + s1.toUpperCase());
        System.out.println("Lowercase: " + s2.toLowerCase());
        System.out.println("Substring: " + s1.substring(1, 4));
        System.out.println("Char at index 1: " + s1.charAt(1));
        System.out.println("Equals: " + s1.equals("Hello"));
```

```
        System.out.println("Compare: " + s1.compareTo(s2));
        System.out.println("Contains 'ell': " + s1.contains("ell"));
    }
}
```

15.Write a JAVA program to implement the concept of exception handling by creating user defined exceptions.

```
import java.util.Scanner;

class InvalidAgeException extends Exception {
    public InvalidAgeException(String msg) {
        super(msg);
    }
}

public class UserDefinedExceptionDemo {

    public static void validate(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age must be 18 or above");
        }
        System.out.println("Valid age: " + age);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("=== USER DEFINED EXCEPTION ===\n");

        System.out.print("Enter age: ");
        int age = sc.nextInt();

        try {
            validate(age);
        } catch (InvalidAgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }

        sc.close();
    }
}
```

16.Write a JAVA program using synchronized threads, which demonstrates producer consumer concept.

```
class Buffer {
    int data;
    boolean available = false;

    synchronized void produce(int value) throws InterruptedException {
        while (available) {
            wait();
        }
        data = value;
        available = true;
        System.out.println("Produced: " + value);
        notify();
    }

    synchronized void consume() throws InterruptedException {
        while (!available) {
            wait();
        }
        System.out.println("Consumed: " + data);
        available = false;
        notify();
    }
}

class Producer extends Thread {
    Buffer buffer;

    Producer(Buffer b) {
        buffer = b;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            try {
                buffer.produce(i);
                Thread.sleep(500);
            } catch (Exception e) {}
        }
    }
}
```

```
class Consumer extends Thread {
   Buffer buffer;

   Consumer(Buffer b) {
      buffer = b;
   }

   public void run() {
      for (int i = 1; i <= 5; i++) {
         try {
            buffer.consume();
            Thread.sleep(500);
         } catch (Exception e) {}
      }
   }
}

public class ProducerConsumerDemo {
   public static void main(String[] args) {
      System.out.println("=== PRODUCER CONSUMER ===\n");

      Buffer b = new Buffer();
      Producer p = new Producer(b);
      Consumer c = new Consumer(b);

      p.start();
      c.start();
   }
}
```

17.Write a JAVA program that creates three threads. First thread displays "Good Morning" every one second, second thread displays "Hello" every two seconds and the third thread displays "Welcome" every three seconds.

```
class Thread1 extends Thread {
   public void run() {
      try {
         while (true) {
            System.out.println("Good Morning");
            Thread.sleep(1000);
         }
      } catch (Exception e) {}
   }
```

```
}

class Thread2 extends Thread {
    public void run() {
        try {
            while (true) {
                System.out.println("Hello");
                Thread.sleep(2000);
            }
        } catch (Exception e) {}
    }
}

class Thread3 extends Thread {
    public void run() {
        try {
            while (true) {
                System.out.println("Welcome");
                Thread.sleep(3000);
            }
        } catch (Exception e) {}
    }
}

public class MultiThreadDemo {
    public static void main(String[] args) {
        System.out.println("=== MULTI-THREADING ===\n");

        Thread1 t1 = new Thread1();
        Thread2 t2 = new Thread2();
        Thread3 t3 = new Thread3();

        t1.start();
        t2.start();
        t3.start();

        // Run for 10 seconds then exit (demo only)
        try {
            Thread.sleep(10000);
        } catch (Exception e) {}

        System.exit(0);
    }
```

```
}
```

18.Write a JAVA program which uses FileInputStream / FileOutputStream Classes.

```
import java.io.*;

public class FileStreamDemo {
   public static void main(String[] args) {
      System.out.println("=== FILE STREAM ===\n");

      String data = "Hello, this is a sample text!";
      String filename = "sample.txt";

      // Write to file
      try (FileOutputStream fos = new FileOutputStream(filename)) {
         byte[] bytes = data.getBytes();
         fos.write(bytes);
         System.out.println("Data written to file: " + filename);
      } catch (IOException e) {
         System.out.println("Error writing: " + e.getMessage());
      }

      // Read from file
      try (FileInputStream fis = new FileInputStream(filename)) {
         int ch;
         System.out.print("Data read from file: ");
         while ((ch = fis.read()) != -1) {
            System.out.print((char) ch);
         }
         System.out.println();
      } catch (IOException e) {
         System.out.println("Error reading: " + e.getMessage());
      }
   }
}
```

19.Write a JAVA program to list all the files in a directory including the files present in all its subdirectories.

```
import java.io.*;

public class ListDirectoryFiles {
```

```
    public static void listFiles(File dir, String indent) {
        File[] files = dir.listFiles();

        if (files == null) return;

        for (File file : files) {
            if (file.isDirectory()) {
                System.out.println(indent + "[DIR] " + file.getName());
                listFiles(file, indent + "  ");
            } else {
                System.out.println(indent + "[FILE] " + file.getName());
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("=== LIST DIRECTORY FILES ===\n");

        String path = "."; // Current directory

        File dir = new File(path);
        System.out.println("Listing files in: " + dir.getAbsolutePath());
        System.out.println("---------------------------------------");

        listFiles(dir, "");
    }
}
```

20.Write a JAVA program to demonstrate the life cycle of applet.

```
import java.applet.Applet;
import java.awt.*;

/*
<applet code="AppletLifeCycle.class" width="400" height="300">
</applet>
*/

public class AppletLifeCycle extends Applet {
   String msg = "";

   public void init() {
      msg += "init() called. ";
      setBackground(Color.yellow);
      System.out.println("init() called");
   }

   public void start() {
      msg += "start() called. ";
      System.out.println("start() called");
   }

   public void paint(Graphics g) {
      msg += "paint() called.";
      g.drawString("Applet Life Cycle", 50, 50);
      g.drawString(msg, 50, 100);
      System.out.println("paint() called");
   }
```

```
    public void stop() {
        System.out.println("stop() called");
    }

    public void destroy() {
        System.out.println("destroy() called");
    }
}
```